SQL Database Project Report

Title: Relational Database Design for Pharmacy Inventory Management System

Introduction:

The project obtains designing and implementation of a relational database system containing 1040 rows, 50 rows, and 5 rows in *product_purchase_details*, *product_information*, and *product_category* tables respectively. The database models an inventory management system for a pharmacy business. The system consists of three related tables and the data has been generated on real-time inputs and with the help of python.

Data Generation:

The idea has been inspired by the *PMS ERP of ARACO Diagnostic & Consultation Center, Bangladesh* which was made by me. The names of the medicines (products) are completely authentic alongside other data from the tables.

Data Generation Process:

1. **Category Table:**
    - A predefined list of category names was created based on general information for the relevant niche.
    - Each category was assigned a unique *category_id*.
    - The '*status*' has been set '1' for being available to show in the ERP system and '0' has been set to hide specific categories from the system.

2. **Product Table:**
    - Multiple products were generated for each category.
    - Product names have been authentic and also been inserted over the time.
    - Price values were generated based on the price in the marketplace and having touch of real-time data.
    - Expiry duration (*expiry_date)* was generated based on real-time data.

3. **Purchase Details Table:**
    - Multiple purchase entries were generated for each product.
    - Quantities, discounts, VAT, and unit rates were generated using realistic data insertion.
    - caution_level was assigned based on suppliers' comments and from the internet, and has been set based on ENUM values: (*'low', 'normal', 'high'*).

All data was generated based on real-time work and no external datasets were downloaded or used. The final dataset was exported as SQL insert statements and structured into relational tables.

Tools Used:

- Python
- pandas
- random module
- numpy

Schema Description (Textual):

The database consists of three tables. Such as-

1. **product_category**

Primary Key: *category_id*
Attributes:
- *category_id* (INT)
- *category_name* (VARCHAR)
- *status* (INT/binary)

Functions:
This table stores product categories and each category surely has multiple products within it.
Relationship: One-to-Many with ***product_information.***

2. **product_information**
Primary Key: *product_id*
Foreign Key: *category_id* → product_category(*category_id*)
Attributes:
- *product_id* (INT)
- *product_name* (VARCHAR)
- *category_id* (INT)
- *price* (DECIMAL)
- *expiry_days* (DATE)
- *status* (INT/binary)

Functions:
This table stores usual product details. Each product belongs to one category.
Relationship:
- Many-to-One with ***product_category***
- One-to-Many with ***product_purchase_details***

3. **product_purchase_details**
Primary Key: *id*
Foreign Key: *product_id* → product_information(*product_id*)
Compound Key: *product_id*
Attributes:
- *id* (INT)
- *product_id* (INT)
- *expire_date* (DATE)
- *quantity* (INT)
- *unit_rate* (DECIMAL)
- *total_amount* (DECIMAL)
- *discount* (DECIMAL)
- *single_vat* (DECIMAL)
- *caution_level* (ENUM: ‘low’, ‘normal’, ‘high’)

Functions:
This table stores purchase records.
Relationship:
- Many-to-One with ***product_information***

Justification for Separate Tables:

The generated database was normalized initially to reduce redundancy and maintain data integrity. The tables are correlated partially with necessary data but do serve separate milestones.
product_category is separate because it:

- avoids repeating category names for each product and maintains 1NF and 2NF.

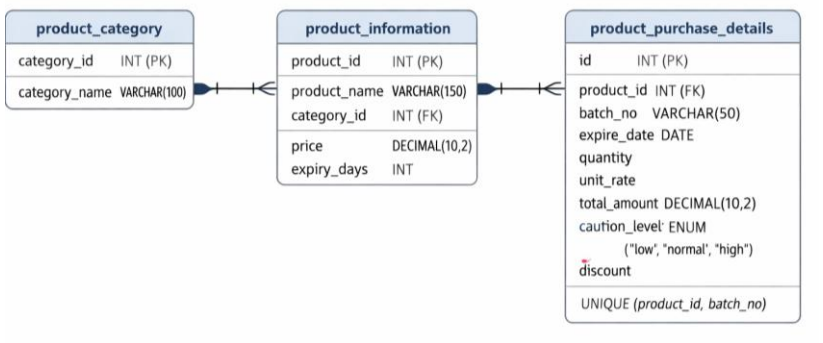
product_information is separate because it:

- stores product attributes free from purchase transactions

product_purchase_details is separate because it:

- supports composite keys to prevent duplicate data.

ER Diagram:



Data Type Classification:

- **Nominal:** product_name, category_name
- **Ordinal:** caution_level (low < normal < high)
- **Interval:** expire_date
- **Ratio:** quantity, price, unit_rate, discount, VAT

Ethical and Data Privacy Considerations:

The dataset may be inspired but is fully synthetic and assured of not containing any real personal or sensitive information. Before the work the notes on this regard have been taken as:

- No real customer data will be used.
- No personal identifiers will be used.
- Data will be generated randomly to avoid bias.
- No external copyrighted dataset will be downloaded.
- The work may be based on generating data but will have some touch of real-time public information.

Thus, this work does not raise any privacy concerns. The use of synthetic data assures compliance with ethical data usage standards.

Conclusion:

The project successfully implements a relational database using SQLite with three normalized tables, with appropriate primary and foreign keys and a composite key. The database contains more than 1000 records and deals with nominal, ordinal, interval, and ratio scales as required. All data is synthetically created to comply with ethical standards. The final database is fully functional and satisfies all requirements of the assignment.

```
C:\sqlite>sqlite3 inventory.db
SQLite version 3.51.2 2026-01-09 17:27:48
Enter ".help" for usage hints.
sqlite> PRAGMA foreign_keys = ON;
sqlite> .read pms.sql
sqlite> .tables
product_category          product_information       product_purchase_details
sqlite> SELECT COUNT(*) FROM product_purchase_details;
1040
sqlite>
```